视频编码测试用例说明书

版本号:
发布日期: 2021-01-18

版本历史

版本号	日期	制/修订人	内容描述

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关术语 1

2 vencoderDemo 的使用 **2**
2.1 vencoderDemo 的用途和用法 2

3 vencoderDemo 流程及关键函数接口说明 **4**
3.1 vencoderDemo 流程 4
3.2 关键函数接口说明 4
3.2.1 Vencoder 关键函数接口说明 4
3.2.1.1 VideoEncCreate 5
3.2.1.2 VideoEncInit 5
3.2.1.3 AllocInputBuffer 6
3.2.1.4 GetOneAllocInputBuffer 6
3.2.1.5 AddOneInputBuffer 7
3.2.1.6 VideoEncodeOneFrame 7
3.2.1.7 AlreadyUsedInputBuffer 8
3.2.1.8 ReturnOneAllocInputBuffer 8
3.2.1.9 GetOneBitstreamFrame 9
3.2.1.10 VideoEncodeOneFrame 9
3.2.1.11 FreeOneBitStreamFrame 10
3.2.1.12 VideoEncDestroy 10

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

2-1 3
3-1 5
3-2 5
3-3 6
3-4 6
3-5 7
3-6 7
3-7 8
3-8 8
3-9 9
3-10 9
3-11 10
3-12 10

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 编写目的

为了让多媒体开发人员熟悉视频编码流程，实现多媒体视频编码功能定制和简单调试。

1.2 适用范围

本视频编码测试用例说明适用于全志科技 Android 及 linux 系统产品。

1.3 相关术语

- CedarC：全志多媒体视频编解码驱动以及 openMAX IL 层实现。
- Encoder：音频，视频，字幕的编码器。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 vencoderDemo 的使用

该解码测试用例可以在 linux 下和 android 下编译使用，android 下对应的编译文件中的目录为`framework/av/media/libcedarc/demo/vencoderDemo/`;linux 下的主要跟 cedarc 的路径相关，例如 t5 linuxR 版本上的路径为 longan/platform/framework/libcedarc/demo/vencoderDemo，linux 下的也可直接编译使用。以下操作以 android 系统为例，linux 系统操作一致。

编译成功后在目录`\out\target\product\mercury-demo\system\bin` 生成名为 vencoderDemo 的可执行文件，将 vencoderDemo 推入小机`system/bin/`目录下，给 777 权限，命令如下：

```
adb root
adb remount
adb push testEnc /system/bin/vencoderDemo
adb shell chmod 777 /system/bin/vencoderDemo
```

2.1 vencoderDemo 的用途和用法

vencoderDemo 视频编码功能测试 demo。用于对输入 YUV 数据进行编码，并保存编码后的裸数据。保存的裸数据可 adb pull 到 pc 端，使用 ffmpeg 打开播放或者其他可以播放裸数据的应用播放，以此来确认编码数据的正确性。

执行 vencoderDemo 时常用的配置参数及说明如下：

- -h: “–help” 打印参数说明
- -s: “–srcsize” 输入源图像分辨率:1080,720,480，其它分辨率需要在代码中自行修改；
- -d: “–dstsize” 输出编码视频分辨率:1080,720,480，其它分辨率需要在代码中自行修改；
- -i: “–input” 输入源图像路径
- -o: “–output” 输出视频路径
- -n: “–encode_frame_num” 编码帧数
- -f: “–encode_format” 编码模式 0：h264; 1: jpeg; 3: h265
- -if: “–input_format” 输入源格式 0:VENC_PIXEL_YUV420SP; 1:VENC_PIXEL_YVU420SP; 2:VENC_PIXEL_YUV420P; 3:VENC_PIXEL_YVU420P

命令如下，将分辨率为 720p 格式为 YUV420P 的/sdcard/720p_10.yuv 图像编码，输出视频分辨率设置为 720p，一共编码 10 帧，编码输出的视频格式为 h264，保存在`/sdcard/camera/720.h264`，若-if 未配置，输入图像默认像素格式为 YUV420P。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
adb shell
su
setenforce 0 //为了保证输出目录具备写文件权限
cd sdcard
mkdir camera
vencoderDemo -i /sdcard/720p_10.yuv -o /sdcard/camera/720.h264 -s 720 -d 720 -n 10 -f 0 -if
     2
```

具体操作如图所示。

```
cupid-p2:/ # su
su
cupid-p2:/ # setenforce 0
setenforce 0
cupid-p2:/ # cd sdcard
cd sdcard
cupid-p2:/sdcard # mkdir camera
mkdir camera
cupid-p2:/sdcard # vencoderDemo -i /sdcard/720p_10.yuv -o /sdcard/camera/720.h264 -s 720 -d 720 -n 10 -f 0 -if 2
.yuv -o /sdcard/camera/720.h264 -s 720 -d 720 -n 10 -f 0 -if 2                <
*****************************************************************
*******test begin time[130s] *******
*****************************************************************

**************[0] cycle demo start time[130s]**************
******************************
get input file: /sdcard/720p_10.yuv
get output file: /sdcard/camera/720.h264
get src_size: 1280x720
get dst_size: 1280x720
encode:10 frames
encode_format:0 0:h264,1:jpeg,3:h265,5: isp
input_format:2 0:VENC_PIXEL_YUV420SP, 1:VENC_PIXEL_YUV420SP, 2:VENC_PIXEL_YUV420P, 3:VENC_PIXEL_YUV420P
******************************
get icon_hdle error
output file is saved:/sdcard/camera/720.h264
*******[0]cycle demo end time[130s] cur_cycle_time[0s]******


*****************************************************************
*******test end time[130s] test_total_time[0s]*******
*****************************************************************
cupid-p2:/sdcard # cd camera
cd camera
cupid-p2:/sdcard/camera # ls
ls
720.h264
cupid-p2:/sdcard/camera #
```

图 2-1

将/sdcard/camera/中的 720.h264 adb pull 到 pc 端，使用 ffplay 播放编码后的数据，确认编码正确性。

3 vencoderDemo 流程及关键函数接口说明

3.1 vencoderDemo 流程

vencoderDemo 接口调用流程见附件《vencoderDemo 流程.pdf》

demo 主函数获取配置参数后创建 ChannelThread 线程，ChannelThread 线程首先分析配置参数 ParserArgument（指定输入输出路径、输入输出分辨率、编码帧数等配置参数），然后进行编码操作，编码结束后 ChannelThread 线程退出，主函数结束。

ChannelThread 线程主要流程如下：

1. ParseArgument 分析编码参数并赋值
2. 获取 mem 函数指针，用于操作 buffer，编码器内部函数使用
3. CdcMemOpen
4. setEncParam 设置参数到编码器
5. VideoEncInit 初始化编码器
6. AllocInputBuffer 申请存放编码前图像帧的缓存
7. GetOneAllocInputBuffer 获取申请的图像帧缓存 inputBuffer 地址
8. 将输入源文件的一帧按照分辨率读取到 inputBuffer 中
9. AddOneInputBuffer 将 inputBuffer 数据送入编码器
10. VideoEncodeOneFrame 编码
11. AlreadyUsedInputBuffer 获取已经编码结束的输入图像帧
12. ReturnOneAllocInputBuffer 归还已经编码结束不再使用的图像帧 buffer
13. GetOneBitstreamFrame 获取编码后的码流
14. 将码流写入要输出的文件中
15. FreeOneBitStreamFrame 归还已写入文件中的码流 buffer
16. VideoEncDestroy 编码全部结束后销毁编码器，释放软硬件资源
17. CdcMemClose

3.2 关键函数接口说明

3.2.1 Vencoder 关键函数接口说明

Vencoder：Cedarc 视频编码器的接口。路径/android/frameworks/av/media/libcedarc/vencoder/，demo 中编码关键接口函数如下。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.1 VideoEncCreate

函数原型	VideoEncoder* VideoEncCreate(VENC_CODEC_TYPE eCodecType);
功能	创建一个视频编码器
参数	eCodecType：创建的编码器 codec 类型
返回值	成功：视频编码器指针；失败：NULL
调用说明	视频编码器支持创建多个编码器，支持多路编码

图 3-1

3.2.1.2 VideoEncInit

函数原型	int VideoEncInit(VideoEncoder* pEncoder, VencBaseConfig* pConfig);
功能	初始化视频编码器
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pConfig：编码器基本初始化信息，包括是否做 scaler，颜色格式等
返回值	成功：0；失败：-1
调用说明	pConfig：编码器基本初始化信息： 1. nInputWidth：输入图像帧的宽度，以像素为单位 2. nInputHeight：输入图像帧的高度，以像素为单位 3. nDstWidth：编码前对输入图像做 scale 后的宽度，以像素为单位，如果不需要做 scale，nDstWidth 的值保持和 nInputWidth 一致 4. nDstHeight：编码前对输入图像做 scale 后的高度，以像素为单位，如果不需要做 scale，nDstHeight 的值保持和 nInputHeight 一致 5. nStride：输入图像帧在内存中的行款宽，以像素为单位，编码器要求 nStride 必须 16 对齐 6. eInputFormat：输入的颜色格式

图 3-2

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.3 AllocInputBuffer

函数原型	int AllocInputBuffer(VideoEncoder* pEncoder, VencAllocateBufferParam *pBufferParam);
功能	通过 vencoder 申请输入图像帧 buffer
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pBufferParam：指定申请 buffer 的格式和 size
返回值	成功：0；失败：-1
调用说明	1. 当需要由编码器来提供输入图像帧的 buffer 时，由此接口来申请图像帧 buffer 2. 当外部模块有自己的 buffer 管理模块，并且所使用的 buffer 为物理连续的 buffer 的时候，从效率上考虑可以不适用此接口来申请输入图像帧 buffer，可以直接把相应的 buffer 物理地址配给 VE，从而可以少一次数据 copy

图 3-3

3.2.1.4 GetOneAllocInputBuffer

函数原型	int GetOneAllocInputBuffer(VideoEncoder* pEncoder, VencInputBuffer* pInputbuffer);
功能	获取由 AllocInputBuffer 申请的输入图像帧
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pInputbuffer：输出参数，获取由 AllocInputBuffer 申请的输入图像帧 buffer
返回值	成功：0；失败：-1
调用说明	pInputbuffer 的相应变量的说明： 1. nID：用来区分不同的 buffer 2. nPts：当前图像帧的时间戳 3. pAddrPhyY：当前图像帧 Y 分量的物理地址，配给硬件使用 4. pAddrPhyC：当前图像帧 C 分量的物理地址，配给硬件使用 5. pAddrVirY：当前图像帧 Y 分量的虚拟地址，可由 CPU 来搬移图像数据到此 buffer 6. pAddrVirC：当前图像帧 C 分量的虚拟地址，可由 CPU 来搬移图像数据到此 buffer

图 3-4

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.5 AddOneInputBuffer

函数原型	int AddOneInputBuffer(VideoEncoder* pEncoder, VencInputBuffer* pInputbuffer);
功能	添加输入图像帧到编码器
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pInputbuffer：输入图像帧 buffer
返回值	成功：0；失败：-1
调用说明	pInputbuffer 的来源可以是 AllocInputBuffer 申请的输入图像帧 buffer，也可以是外部模块提供

图 3-5

3.2.1.6 VideoEncodeOneFrame

函数原型	int VideoEncodeOneFrame(VideoEncoder* pEncoder);
功能	编码一帧图像
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针
返回值	VENC_RESULT_ERROR（-1）：编码出错 VENC_RESULT_OK（0）：编码成功 VENC_RESULT_NO_FRAME_BUFFER（1）：无法获取输入帧 VENC_RESULT_BITSTREAM_IS_FULL（2）：输入码流 buffer 溢出
调用说明	NA

图 3-6

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.7 AlreadyUsedInputBuffer

函数原型	int AlreadyUsedInputBuffer(VideoEncoder* pEncoder, VencInputBuffer* pBuffer);
功能	获取 VideoEncodeOneFrame 已经使用过的输入图像帧
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pBuffer：输出参数，图像帧 buffer
返回值	成功：0；失败：-1
调用说明	NA

图 3-7

3.2.1.8 ReturnOneAllocInputBuffer

函数原型	int ReturnOneAllocInputBuffer(VideoEncoder* pEncoder, VencInputBuffer *pInputbuffer);
功能	还回由 AllocInputBuffer 申请的输入图像帧 buffer
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pInputbuffer：由 AllocInputBuffer 申请的输入图像帧 buffer
返回值	成功：0；失败：-1
调用说明	NA

图 3-8

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.9 GetOneBitstreamFrame

函数原型	int GetOneBitstreamFrame(VideoEncoder* pEncoder, VencOutputBuffer* pBuffer);
功能	获取有效的输出码流 buffer
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pBuffer：输出参数，输出码流 buffer
返回值	成功：0；失败：-1
调用说明	pBuffer 中结构体变量说明： 1. nID：用来识别不同的 buffer 2. nPts：编码器不处理时间戳信息，输出 buffer 中 pts 对应相应输入 buffer 中 pts 3. nSize0：输出码流第一部分的大小 4. nSize1：输出码流第二部分的大小 5. pData0：输出码流第一部分的地址 6. pData1：输出码流第二部分的地址 输出的一笔码流可能由两部分组成：nSize0 第一部分大小，nSize1 第二部分大小 nSize0 一定大于 0，当 nSize1=0 时，输出码流只有地址 pData0 中，当 nSize1>0，输出码流由两部分组成，第一部分在 pData0 中，第二部分在 pData1 中，此时需要外部应用程序把这两部分数据组合成一帧

图 3-9

3.2.1.10 VideoEncodeOneFrame

函数原型	int VideoEncodeOneFrame(VideoEncoder* pEncoder);
功能	编码一帧图像
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针
返回值	VENC_RESULT_ERROR（-1）：编码出错 VENC_RESULT_OK（0）：编码成功 VENC_RESULT_NO_FRAME_BUFFER（1）：无法获取输入帧 VENC_RESULT_BITSTREAM_IS_FULL（2）：输入码流 buffer 溢出
调用说明	NA

图 3-10

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.1.11 FreeOneBitStreamFrame

函数原型	int FreeOneBitStreamFrame(VideoEncoder* pEncoder, VencOutputBuffer* pBuffer);
功能	归还输出码流 buffer
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针 pBuffer：由 GetOneBitstreamFrame 获取到的输出码流 buffer
返回值	成功：0；失败：-1
调用说明	NA

图 3-11

3.2.1.12 VideoEncDestroy

函数原型	void VideoEncDestroy (VideoEncoder* pEncoder);
功能	销毁一个视频编码器，释放相关软硬件资源
参数	pEncoder：通过 VideoEncCreate 函数创建的视频编码器指针
返回值	
调用说明	NA

图 3-12

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2021 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。